



DOCUMENTAÇÃO COMPLETA DO PROJETO

PAINELÉ



VISÃO GERAL DO PROJETO

Painelé é um sistema completo de gestão empresarial desenvolvido com arquitetura full-stack moderna, oferecendo funcionalidades para controle de vendas, produtos, despesas, funcionários e relatórios.



ARQUITETURA GERAL

- **Frontend:** React.js com Vite e TailwindCSS
- **Backend:** Node.js com Express.js
- **Banco de Dados:** PostgreSQL com Prisma ORM
- **Autenticação:** JWT (JSON Web Tokens)
- **Pagamentos:** Integração com Mercado Pago
- **Email:** Nodemailer para recuperação de senha



TECNOLOGIAS UTILIZADAS

Backend (Paineleback)

```
{  "runtime": "Node.js",  "framework": "Express.js",  "orm": "Prisma",  "database": "PostgreSQL",  "authentication": "JWT + bcrypt",  "payment": "Mercado Pago",  "email": "Nodemailer",  "security": "Helmet + CORS",  "development": "Nodemon"}
```

Frontend (Painelefront)

```
{  "framework": "React 18.3.1",  "bundler": "Vite",  "styling": "TailwindCSS 4.1.4",  "routing": "React Router DOM 7.5.1",  "http": "Axios",  "charts": "Recharts",  "icons": "Lucide React",  "notifications": "React Toastify",  "ui_components": "Radix UI"}
```

ESTRUTURA DO BANCO DE DADOS

MODELOS PRINCIPAIS

User (Usuário Principal)

```
model User {
  id          String      @id @default(uuid())
  nome        String
  email       String      @unique
  senha       String
  telefone    String?
  tipo        String      @default("PRINCIPAL")
  assinatura   String      @default("gratuito")
  planoExpiraEm DateTime?
  resetToken   String?
  resetTokenExpiry DateTime?
  criadoEm     DateTime    @default(now())

  // Relacionamentos
  vendas       Venda[]     @relation("VendasDoUsuario")
  produtos     Produto[]   @relation("ProdutosDoUsuario")
  despesas     Despesa[]    @relation("DespesasDoUsuario")
  assinaturas  Assinatura[] @relation("AssinaturasDoUsuario")
  pagamentos   Pagamento[] @relation("PagamentosDoUsuario")
  funcionarios  Funcionario[]

}
```

Funcionario

```
model Funcionario {
  id      String  @id @default(uuid())
  nome    String
  email   String  @unique
  senha   String
  ativo   Boolean @default(true)
  criadoEm DateTime @default(now())
  usuarioId String

  // Relacionamentos
  User      User      @relation(fields: [usuarioId], references: [id], onDelete: Cascade)
  Venda     Venda[]

}
```

Produto

```
model Produto {
  id      String      @id @default(uuid())
  nome    String
```

```

    valor      Float
    quantidade Int
    estoque    Int          // estoque inicial
    vendidos   Int          @default(0)
    categoria  String
    usuarioId  String

    // Relacionamentos
    usuario    User          @relation("ProdutosDoUsuario", fields: [usuarioId], references:
[id], onDelete: Cascade)
    itensVenda ItemVenda[]
}

```

Venda

```

model Venda {
    id          String      @id @default(uuid())
    tipo        String
    usuarioId   String
    funcionarioId String?
    identificador String?
    createdAt   DateTime    @default(now())
    finalizada  Boolean      @default(false)

    // Relacionamentos
    usuario     User        @relation("VendasDoUsuario", fields: [usuarioId], references:
[id], onDelete: Cascade)
    funcionario  Funcionario? @relation(fields: [funcionarioId], references: [id])
    itensVenda  ItemVenda[]
}

```

Despesa

```

model Despesa {
    id          String      @id @default(uuid())
    descricao   String
    valor       Float
    categoria   String
    data        DateTime    @default(now())
    usuarioId   String
    criadoEm    DateTime    @default(now())
    atualizadoEm DateTime    @updatedAt

    // Relacionamentos
    usuario     User        @relation("DespesasDoUsuario", fields: [usuarioId], references: [id],
onDelete: Cascade)
}

```

Assinatura

```

model Assinatura {
  id          String          @id @default(uuid())
  usuarioId   String
  plano       PlanoTipo       @default(GRATUITO)
  status      StatusAssinatura @default(ATIVA)
  dataInicio  DateTime        @default(now())
  dataFim     DateTime?
  precoMensal Float?
  mercadoPagoId String?       @unique

  // Relacionamentos
  usuario      User           @relation("AssinaturasDoUsuario", fields: [usuarioId],
references: [id])
  pagamentos   Pagamento[]    @relation("PagamentosAssinatura")
}

```

RELACIONAMENTOS

- **User** → **Funcionario**: 1:N (Um usuário pode ter vários funcionários)
- **User** → **Produto**: 1:N (Um usuário pode ter vários produtos)
- **User** → **Venda**: 1:N (Um usuário pode ter várias vendas)
- **User** → **Despesa**: 1:N (Um usuário pode ter várias despesas)
- **Funcionario** → **Venda**: 1:N (Um funcionário pode fazer várias vendas)
- **Produto** → **ItemVenda**: 1:N (Um produto pode estar em vários itens de venda)
- **Venda** → **ItemVenda**: 1:N (Uma venda pode ter vários itens)

ESTRUTURA DO BACKEND

Organização de Pastas

```

Paineleback/
├── src/
│   ├── controllers/    # Controladores das rotas
│   ├── middlewares/    # Middlewares de autenticação e validação
│   ├── routes/         # Definição das rotas da API
│   ├── services/       # Lógica de negócio
│   ├── utils/          # Utilitários e helpers
│   └── prisma/         # Schema e configurações do Prisma
├── prisma/             # Schema principal do Prisma
├── server.js           # Arquivo principal do servidor
└── package.json        # Dependências e scripts

```

CONTROLLERS

auth.controller.js

- **register**: Cadastro de novos usuários
- **login**: Autenticação de usuários e funcionários
- **logout**: Encerramento de sessão
- **forgotPassword**: Recuperação de senha
- **resetPassword**: Redefinição de senha

- **checkStatus:** Verificação de status de autenticação

dashboard.controller.js

- Métricas e estatísticas do negócio
- Resumos de vendas, produtos e despesas
- Dados para gráficos e relatórios

venda.controller.js

- **criarVenda:** Criação de novas vendas
- **listarVendas:** Listagem de vendas do usuário
- **atualizarVenda:** Atualização de vendas existentes
- **encerrarVenda:** Finalização de vendas
- **excluirVenda:** Exclusão de vendas

produto.controller.js

- **criarProduto:** Cadastro de novos produtos
- **listarProdutos:** Listagem de produtos do usuário
- **atualizarProduto:** Atualização de produtos
- **excluirProduto:** Exclusão de produtos
- **buscarPorId:** Busca de produto específico

despesa.controller.js

- **criarDespesa:** Cadastro de novas despesas
- **listarDespesas:** Listagem de despesas do usuário
- **atualizarDespesa:** Atualização de despesas
- **excluirDespesa:** Exclusão de despesas

funcionarios.controller.js

- **criar:** Cadastro de novos funcionários
- **listar:** Listagem de funcionários do usuário
- **buscarPorId:** Busca de funcionário específico
- **atualizar:** Atualização de dados do funcionário
- **deletar:** Exclusão de funcionários

MIDDLEWARES

verificaToken.js

```
// Verifica se o token JWT é válido
// Extrai informações do usuário do token
// Adiciona dados do usuário à requisição (req.user)
```

verificaPermissao.js

```
// Verifica se o usuário tem permissão para acessar o recurso
// Valida planos de assinatura
// Controla acesso a funcionalidades premium
```

SERVICES

auth.service.js

```
// Lógica de autenticação para usuários e funcionários
// Geração e validação de tokens JWT
// Criptografia de senhas com bcrypt
// Recuperação e redefinição de senhas
```

email.service.js

```
// Envio de emails de recuperação de senha
// Templates de email personalizados
// Configuração SMTP
```

ESTRUTURA DO FRONTEND

Organização de Componentes

```
Painelefront/src/
├── components/
│   ├── Auth/           # Componentes de autenticação
│   ├── Layout/         # Layout e navegação
│   ├── Login/          # Telas de login e cadastro
│   ├── Produtos/       # Gestão de produtos
│   ├── Vender/         # Sistema de vendas
│   ├── Despesas/       # Gestão de despesas
│   ├── Usuarios/       # Gestão de usuários e funcionários
│   ├── Relatorio/      # Relatórios e dashboards
│   ├── Assinatura/     # Gestão de planos
│   └── ui/             # Componentes de interface
├── pages/              # Páginas principais
├── services/           # Serviços de API
├── hooks/              # Hooks customizados
├── utils/              # Utilitários
└── lib/                # Configurações e bibliotecas
```

PÁGINAS PRINCIPAIS

Home.jsx

- Dashboard principal com métricas
- Resumo de vendas, produtos e despesas
- Gráficos e estatísticas

Login.jsx

- Formulário de autenticação
- Suporte a login de usuários e funcionários
- Links para cadastro e recuperação de senha

Products.jsx

- Listagem de produtos
- Formulários de cadastro e edição

- Controle de estoque

Selling.jsx

- Interface de vendas
- Carrinho de compras
- Finalização de vendas

Despesas.jsx

- Listagem de despesas
- Formulários de cadastro e edição
- Categorização de despesas

Usuarios.jsx

- Gestão de funcionários
- Controle de permissões
- Status ativo/inativo

Relatorio.jsx

- Relatórios detalhados
- Gráficos de performance
- Exportação de dados

COMPONENTES DE UI

Layout/Navbar.jsx

```
// Barra de navegação principal
// Menu responsivo
// Logout e informações do usuário
```

ui/toast.jsx

```
// Sistema de notificações
// Feedback visual para ações
// Integração com React Toastify
```

API ENDPOINTS

Autenticação (/api/auth)

POST	/register	# Cadastro de usuário
POST	/login	# Login de usuário/funcionário
POST	/logout	# Logout
POST	/forgot-password	# Recuperação de senha
POST	/reset-password	# Redefinição de senha
GET	/status	# Status de autenticação

Produtos (/api/produtos)

GET	/	# Listar produtos
POST	/	# Criar produto
GET	/:id	# Buscar produto por ID
PUT	/:id	# Atualizar produto
DELETE	/:id	# Excluir produto

Vendas (/api/vendas)

GET	/	# Listar vendas
POST	/	# Criar venda
GET	/:id	# Buscar venda por ID
PUT	/:id	# Atualizar venda
DELETE	/:id	# Excluir venda
POST	/:id/encerrar	# Encerrar venda

Despesas (/api/despesas)

GET	/	# Listar despesas
POST	/	# Criar despesa
GET	/:id	# Buscar despesa por ID
PUT	/:id	# Atualizar despesa
DELETE	/:id	# Excluir despesa

Funcionários (/api/funcionarios)

GET	/	# Listar funcionários
POST	/	# Criar funcionário
GET	/:id	# Buscar funcionário por ID
PUT	/:id	# Atualizar funcionário
DELETE	/:id	# Excluir funcionário

Dashboard (/api/dashboard)

GET	/	# Métricas gerais
GET	/vendas	# Estatísticas de vendas
GET	/produtos	# Estatísticas de produtos
GET	/despesas	# Estatísticas de despesas

SISTEMA DE AUTENTICAÇÃO

JWT (JSON Web Tokens)

- **Geração:** Após login bem-sucedido
- **Armazenamento:** Cookie httpOnly + localStorage
- **Expiração:** 12 horas
- **Renovação:** Automática em requisições

Tipos de Usuário

1. **PRINCIPAL:** Usuário proprietário da conta

2. **FUNCIONARIO:** Funcionário vinculado ao usuário principal

Middleware de Segurança

```
// verificaToken: Valida JWT em todas as rotas protegidas
// verificaPermissao: Controla acesso baseado no plano
// CORS: Configurado para domínios específicos
// Helmet: Headers de segurança
```

SISTEMA DE PAGAMENTOS

Mercado Pago Integration

- **Planos Disponíveis:**
 - GRATUITO: Funcionalidades básicas
 - BÁSICO: Funcionalidades intermediárias
 - PREMIUM: Funcionalidades avançadas
 - ENTERPRISE: Funcionalidades completas

Fluxo de Pagamento

1. Usuário seleciona plano
2. Redirecionamento para Mercado Pago
3. Processamento do pagamento
4. Webhook de confirmação
5. Ativação do plano

SISTEMA DE EMAIL

Nodemailer Configuration

- **Recuperação de Senha:** Email com token temporário
- **Templates:** HTML personalizados
- **Segurança:** Tokens com expiração

DEPLOY E PRODUÇÃO

Backend (Railway)

```
{
  "platform": "Railway",
  "database": "PostgreSQL",
  "environment": "Node.js",
  "domain": "Custom domain support"
}
```

Frontend (Vercel)

```
{
  "platform": "Vercel",
  "build": "Vite",
  "deployment": "Automatic from Git",
  "domain": "Custom domain support"
}
```

Variáveis de Ambiente

```
# Backend
DATABASE_URL=postgresql://...
JWT_SECRET=your_jwt_secret
MERCADOPAGO_ACCESS_TOKEN=your_mp_token
EMAIL_HOST=smtp.gmail.com
EMAIL_USER=your_email
EMAIL_PASS=your_password

# Frontend
VITE_API_URL=https://your-backend-url.com
```

FLUXOS PRINCIPAIS

Fluxo de Autenticação

1. **Login:** Email/senha → Validação → JWT → Cookie + localStorage
2. **Funcionário:** Email/senha → Busca em Funcionario → Herda plano do usuário principal
3. **Proteção:** Middleware verifica JWT em todas as rotas protegidas

Fluxo de Vendas

1. **Criação:** Seleção de produtos → Carrinho → Finalização
2. **Itens:** Produtos são convertidos em ItemVenda
3. **Estoque:** Atualização automática após venda
4. **Funcionário:** Vendas podem ser associadas a funcionários

Fluxo de Produtos

1. **Cadastro:** Nome, valor, quantidade, categoria
2. **Estoque:** Controle de quantidade disponível
3. **Vendas:** Redução automática do estoque
4. **Relatórios:** Produtos mais vendidos

FUNCIONALIDADES PRINCIPAIS

Implementadas

-  Sistema de autenticação completo
-  Gestão de usuários e funcionários
-  CRUD de produtos com controle de estoque
-  Sistema de vendas com carrinho
-  Gestão de despesas por categoria

-  Dashboard com métricas em tempo real
-  Relatórios e gráficos
-  Sistema de planos e assinaturas
-  Integração com Mercado Pago
-  Recuperação de senha por email
-  Interface responsiva
-  Notificações em tempo real

Em Desenvolvimento

-  Relatórios avançados em PDF
-  Sistema de backup automático
-  Integração com outros gateways de pagamento
-  App mobile

COMANDOS ÚTEIS

Backend

```
# Desenvolvimento
npm run dev

# Produção
npm start

# Prisma
npm run migrate
npx prisma studio
npx prisma generate
```

Frontend

```
# Desenvolvimento
npm run dev

# Build
npm run build

# Preview
npm run preview

# Lint
npm run lint
```

CONSIDERAÇÕES FINAIS

O **Painelê** é um sistema robusto e escalável, desenvolvido com as melhores práticas de desenvolvimento web moderno. A arquitetura modular permite fácil manutenção e expansão de funcionalidades.

Pontos Fortes

- **Segurança:** JWT + bcrypt + middlewares de proteção
- **Performance:** React otimizado + Prisma ORM
- **Escalabilidade:** Arquitetura modular e componentizada
- **UX/UI:** Interface moderna e responsiva
- **Integração:** Mercado Pago + Email + PostgreSQL

Roadmap Futuro

- Implementação de WebSockets para atualizações em tempo real
- Sistema de notificações push
- Integração com APIs de terceiros (contabilidade, estoque)
- Dashboard analytics avançado
- Sistema de backup e restore

Documentação gerada automaticamente

Versão: 1.0

Data: Janeiro 2025

Projeto: Painelê - Sistema de Gestão Empresarial